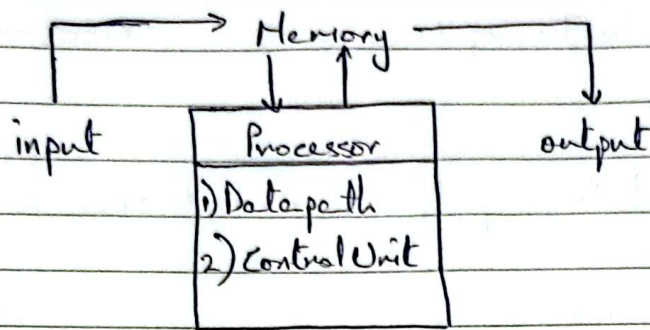


Computer Architecture

Date: _____



-) Memory contains data & instructions.
-) Processor contains the data path & the control unit.
-) Clock in the control unit synchronizes the control unit. At every positive edge of a clock pulse, a different instruction can be executed.
-) Processor executes instructions such as addition of 2 numbers. An instruction can take certain number of clock cycles.
-) Program Counter (PC) keeps track of the instruction to execute next. It stores the address of the next instruction to execute.
-) A compiler converts high level language into Assembly Language, the Assembler converts assembly language into Machine Code for the computer to understand.
-) The Processor fetches & decodes instructions.
-) Response Time: "Time taken for a task to finish."
-) Throughput: "Total number of tasks completed in a certain amount of time."
-) A faster processor will decrease the response time & increases the throughput.
-) Adding more processors keeps the response time same but increases the throughput.

•) Speedup of x relative to y :-

The phrase "X is faster than Y"

$$\therefore \frac{\text{Performance of X}}{\text{Performance of Y}} = n \quad *) n \text{ will be greater than } 1$$

$$*) \text{ Performance of } X = \frac{1}{\text{Execution Time of } X}, \text{ Performance of } Y = \frac{1}{\text{Execution Time of } Y}$$

•) Elapsed Time :-

*) Includes all system overheads to complete a task. Disk accesses, memory accesses etc.

*) Determines system performance.

•) CPU Time (Computation Time) :- "CPU Time means the time the processor is computing on one task, not including time for I/O operations or running other programs."

$$*) \text{ System Performance} = \frac{1}{\text{Elapsed Time of } X}$$

$$*) \text{ CPU Performance} = \frac{1}{\text{CPU Time of } X}$$

$$*) \text{ CPU Time} = \text{Instruction Count} \times \text{Avg. Cycles/Instr.} \times \text{Clock Cycle Time}$$

$$*) \text{ CPU Time} = \frac{\text{Instruction Count} \times \text{Avg. Cycles/Instr.}}{\text{Clock Rate}} \quad *) \text{ Clock rate in Hz}$$

$$*) \text{ Total Cycles} = \text{Instruction Count} \times \text{Avg. Cycles/Instr.}$$

-) Class A: Takes 1 cycle $\rightarrow$ has 6000 instructions
 Class B: Takes 2 cycles $\rightarrow$ has 3000 instructions
 Class C: Takes 5 cycles $\rightarrow$ has 1000 instructions

$$6000 \times 1 = 6000 \text{ cycles}$$

$$3000 \times 2 = 6000 \text{ cycles}$$

$$1000 \times 5 = 5000 \text{ cycles}$$

$$17000 \text{ total cycles}$$

$$\text{Avg. Cycles / instr.} = \frac{17000}{10000} = 1.7 \text{ Avg. Cycles / Instr.}$$

$$= \frac{\text{Total No. of instructions}}{\text{Total No. of cycles}}$$

$$*) \text{ Avg. CPI} = \frac{\text{Total No. of cycles}}{\text{Total No. of instructions}}$$

~~*) CPI =~~

$$*) \text{ CPU clock cycles} = \sum_{i=1}^n \text{Instruction Cycles}_i \times \text{CPI}_i$$

$$*) \text{ CPU Time} = \left[\sum_{i=1}^n \text{IC}_i \times \text{CPI}_i \right] \times \text{Clock Cycle Time}$$

-) Example :-

	A	B	C
CPI	1	2	3

Code Sequence	A	B	C	
1	2	1	2	= 5 instructions
2	4	1	1	= 6 "

For code sequence 1:

$$\text{class A} = 2 \times 1 = 2$$

$$\text{B} = 2 \times 1 = 2$$

$$\text{C} = 3 \times 2 = 6$$

10 cycles

$$\text{Avg CPI} = \frac{10}{8} = 1.25$$

For code Sequence 2:

$$\text{class A} = 4 \times 1 = 4$$

$$\text{B} = 2 \times 1 = 2$$

$$\text{C} = 3 \times 1 = 3$$

9 cycles

$$\text{Avg CPI} = \frac{9}{6} = 1.5$$

*) Code sequence 2 is faster.

→ Example :-

Old Processor : CPU Time = 15 seconds

$$IC = IC_0$$

$$CPI = CPI_0$$

$$\text{Clock Cycle} = \text{Clock Cycle}$$

New Processor : CPU Time = ?

$$IC = 0.6 IC_0$$

$$CPI = 1.1 CPI_0$$

$$\text{Clock Cycle} = \text{Clock Cycle}$$

$$\text{Old} \Rightarrow 15 = IC_0 \times CPI_0 \times \text{Clock Cycle} \rightarrow \text{eq. 1}$$

$$\text{New} \Rightarrow x = 0.6 IC_0 \times 1.1 CPI_0 \times \text{Clock Cycle} \rightarrow \text{eq. 2}$$

$$\text{Clock Cycle} = \frac{15}{IC_0 \times CPI_0}$$

$$\begin{aligned} \text{Clock Cycle} &= \\ x &= \frac{0.6 IC_0 \times 1.1 CPI_0 \times 15}{IC_0 \times CPI_0} \end{aligned}$$

$$x = 0.6 \times 1.1 \times 15 = 9.9 \text{ CPU Time}$$

→ Improving Performance :-

→ Amdahl's Law :-

$$*) \quad \text{Execution Time after Improvement} = \frac{\text{Execution time affected by improvement}}{\text{Amount of improvement}} + \text{Execution time unaffected}$$

~~Example :-~~

* Time unaffected by improvement = Total Time - Time of improvement

-) Example :-

Time needed to improve = 160s

How much to improve = 4 times

Time unaffected = $200 - 160 = 40$

$$\therefore \text{Time improved} = \frac{160}{4} + 40 = 80\text{s}$$

-) Measuring performance through MIPS :

$$*) \text{ MIPS} = \frac{\text{Instruction Count}}{\text{Execution Time} \times 10^6}$$

*) ^{Tells} ~~Performance~~ millions of instructions / second only, does not tell performance

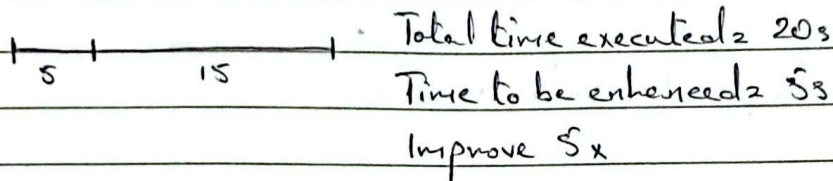
$$*) \text{ MIPS} = \frac{\text{Clock Rate}}{\text{CPI} \times 10^6} = \frac{\text{Clock Rate}}{\text{CPI} \times 10^6}$$

*) Example :

$$*) \text{ Speedup} = \frac{1}{f_e + (1-f_e) S_e}$$

$$*) f_e = \frac{\text{Time to be enhanced}}{\text{Total time executed}}$$

$$*) S_e = \frac{\text{Time to be enhanced}}{\text{Improve } S_x}$$



$$\therefore f_e = \frac{5}{20} = 0.25$$

$$\therefore S_e = 5$$

$$\therefore \text{Speedup} = \frac{1}{0.25 + (1-0.25) 5} = 1.25 \text{ time speedup}$$

*) CPI weighted Average:

CPI	Instruction Mix
1	60% = 0.6
2	18% = 0.18
4	12% = 0.12
8	10% = 0.1

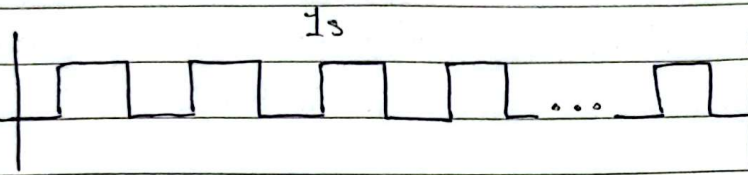
$$\text{Avg. CPI} = (1 \times 0.6) + (2 \times 0.18) + (4 \times 0.12) + (8 \times 0.1) = 2.24$$

$$\text{Clock rate} = 2.4 \text{ GHz}$$

Date: _____

$$\text{MIPS} = \frac{\text{Clock Rate}}{\text{CPI} \times 10^6} \Rightarrow \frac{2.4 \times 10^9}{2.24 \times 10^6} = 1071 \text{ MIPS}$$

↳ Million Instructions / second



- *) 1s takes ≈ 2.4 billion clock cycles
- *) 1 cycle takes 2.24 clock cycles

$$\therefore \frac{2.4 \times 10^9}{2.24 \times 10^6} = 1071 \text{ MIPS}$$

•) Instruction Set Architecture: "ISA are the machine attributes, a system programmer needs to know in order to develop system software or a compiler requires for translation of a high level language (HLL) code into machine language."

- 1) Instruction Set
- 2) Programmer Accessible Registers
- 3) Addressing Modes
- 4) Instruction formats
- 5) Execution Models

→) Goals of an instruction set designer:-

- 1) Make it easy to build the hardware.
- 2) Make it easy to build the compiler.
- 3) Max performance
- 4) Minimize cost
- 5) Minimize energy consumption

CISC

- 1) Large, complex set
- 2) Instructions can vary in length, which allows for complex instructions.

RISC

- 1) Smaller, simpler instruction set
- 2) Fixed length instruction format
- 3) Focuses on reducing the cycles per instruction

•) RISC-V :-

*) Consists of a base + extension

-) Base :

RV32 → Can add 2 32 bit integer / Processes 32 bits / 32 registers

RV64 → Can add 2 64 bit integers / Processes 64 bits / ~~64~~ 32 registers

*) We will work with RV64I.